

SaaS FinOps — Plan de Profesionalización

CSCloudSolutions · Generado 29 de junio 2026 · Documento interno

Objetivo: hoja de ruta priorizada por ROI para llevar el SaaS de su estado actual (multi-tenant Azure-first con tier sales) a producto Enterprise-ready vendible a clientes regulados, con canales self-service, billing automático y diferenciación por features avanzadas FinOps.

1. Tests + CI

Esfuerzo: 5–7 días · **Tier:** 1 (hacer ya) · **ROI:** Alto

Estado actual

- Sin runner de tests, sin tests existentes.
- CI = solo `deploy.yml` (SSH + docker compose).
- TypeScript ya brinda garantías de tipos.

Stack propuesto

- **Vitest** — mejor integración Next.js que Jest, ESM nativo, ~10x más rápido.
- **@testing-library/react** + **jest-dom** para componentes.
- **Playwright** para E2E (un solo flujo crítico, no full coverage).
- **MSW** (Mock Service Worker) para interceptar fetch en tests.

Estructura

```
__tests__/  
  unit/                ← lib/, services/ puros  
    pricingUnits.test.ts  
    fx.test.ts  
    carbonService.test.ts  
    tierLogic.test.ts  
  integration/         ← API routes con DB en memoria/mock  
    api-mcp.test.ts  
    api-fx.test.ts  
  components/          ← React Testing Library  
    PinButton.test.tsx  
    CurrencySelector.test.tsx  
  e2e/                  ← Playwright  
    login-flow.spec.ts  
    onboarding.spec.ts
```

Cobertura objetivo (MVP)

- **80% lib/:** helpers puros, fáciles y críticos (pricingUnits, fx, carbon, tierLogic, requestAuth).

- **60% services/**: mock de pool DB, validar lógica.
- **40% API routes**: smoke tests de respuestas + auth.
- **3 flujos E2E**: login → dashboard, crear budget, ver costos.

CI workflow nuevo `.github/workflows/ci.yml`

```
on: [pull_request, push]
jobs:
  lint:      npm run lint
  typecheck: npx tsc --noEmit
  unit:      npm test -- --coverage
  e2e:       npx playwright test (matrix node 20)
  build:     npm run build (verifica que compila)
```

Sprints

Día	Entregable
1	Vitest setup, primer test verde, config CI
2-3	Tests unit de lib/ (pricingUnits, fx, carbon, tierLogic, requestAuth)
4	Tests de API routes críticos (auth, MCP, fx, billing)
5	Component tests (PinButton, CurrencySelector, ScopeSelector)
6	Playwright + 3 flujos E2E
7	Branch protection, badges, docs

Entregables finales

- 50-80 tests verdes en CI.
- Coverage badge en README.
- PR template con checklist "tests added".
- Branch protection que exige CI verde.

2. SSO SAML

Esfuerzo: 3–5 días (homebrew) · 1–2 días (WorkOS) · **Tier:** 1 · **ROI:** Alto Enterprise

Estado actual

- Auth solo Entra ID (OAuth/MSAL).
- Tabla `Users` con `entra_oid` + `email`.

Por qué SAML

Clientes Enterprise grandes usan Okta, Auth0, OneLogin, Ping, Google Workspace SAML. Sin SAML, cierran negocios alternativos. Es el dealbreaker más citado en RFPs.

Stack

- `@node-saml/passport-saml` (mantenido, soporta Next.js).
- O **WorkOS / Auth0 Enterprise Connections** (managed, ~\$0.05/user). *Recomendado para MVP por time-to-market.*
- Almacenamiento: nueva tabla `TenantSAMLConfig`.

Schema

```
CREATE TABLE TenantSAMLConfig (  
  tenant_id VARCHAR(128) PRIMARY KEY,  
  entity_id VARCHAR(255),  
  sso_url VARCHAR(512),  
  certificate TEXT,  
  attribute_mapping_json JSON,  
  enabled BOOLEAN DEFAULT 0,  
  enforced BOOLEAN DEFAULT 0,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);
```

Endpoints nuevos

- `GET /api/auth/saml/[tenantId]/login` — redirige al IdP.
- `POST /api/auth/saml/[tenantId]/acs` — Assertion Consumer Service.
- `GET /api/auth/saml/[tenantId]/metadata` — XML para configurar IdP.
- `/api/admin/sso` — UI config (upload cert, set URL, attribute mapping).

Flujo

1. Tenant admin va a `/admin/sso`, configura IdP (URL + cert).
2. Genera URL `/api/auth/saml/<tenant>/login` y la comparte con su IdP.

3. Usuario → "Login with SSO" → ingresa email → detectamos dominio → redirige al IdP → callback al ACS → emitimos nuestro JWT.
4. Si `enforced=true` , usuarios del dominio NO pueden hacer login via Entra/local — solo SAML.

Sprints

Día	Entregable
1	Schema + endpoint metadata + UI básica
2	Login init + ACS callback con <code>@node-saml</code>
3	Attribute mapping (email, name, role) + crear/sincronizar User
4	UI admin SSO + tests con SAMLtest.id (idp gratuito de prueba)
5	Doc onboarding para Okta/Auth0/Google Workspace + edge cases (logout SLO)

Alternativa rápida — WorkOS (1-2 días total)

- Una sola API key, ellos manejan los IdPs.
- \$0.05/user/mes (negociable a fixed pricing >100 users).
- Entregás SAML + SCIM provisioning + Directory Sync de paso.
- **Recomendado** si el time-to-market importa más que el costo.

3. Roadmap general — Profesionalización del SaaS

Items priorizados por ROI, segmentados en 4 tiers. Billing se hará con **Paddle** (no Stripe).

🔥 Tier 1 — Hacer YA (próximas 4-6 semanas)

#	Item	Esfuerzo	ROI	Razón
1	Tests + CI	5–7d	Alto	Sin esto no podés escalar el equipo ni vender Enterprise.
2	SSO SAML (WorkOS)	1–2d	Alto	Dealbreaker para Enterprise; quick win con WorkOS.
3	Paddle real end-to-end	3–5d	Crítico	Ya hay scaffolding (<code>/api/webhooks/paddle</code> , <code>/api/checkout</code> , <code>/api/billing/portal</code>); falta upgrade/downgrade/proration completos.
4	Onboarding wizard guiado	4–6d	Alto	Reduce churn early; convierte trial → paid.
5	Notificaciones Slack/Teams/Email	3–4d	Alto	Ya hay <code>notifications.ts</code> (54 LOC); falta Slack/Teams webhook + SMTP/Resend.

🟡 Tier 2 — Próximo trimestre

#	Item	Esfuerzo	ROI	Razón
6	Status page pública	2d	Medio-alto	StatusPage.io o homebrew (<code>/status</code>).
7	Audit log exportable	1–2d	Medio	Ya hay <code>auditService.ts</code> + <code>/api/admin/audit</code> ; falta CSV export + filtros UI.
8	API pública v1 + OpenAPI	5–7d	Medio-alto	Diferencia vs competidores closed; habilita partners.
9	Self-service signup + free trial	4–5d	Crítico para growth	Ya hay <code>demoSession</code> ; falta funnel + nurturing.
10	Showback/Chargeback PDF	4–6d	Alto Enterprise	AKS chargeback ya existe; falta PDF invoice por BU.

🟢 Tier 3 — Cuando haya tracción

#	Item	Esfuerzo	Notas
11	FOCUS 1.x exporter	1–2 sem	Diferenciador Enterprise. Feature A del plan original (diferida).
12	2FA local	3d	Solo si tenés usuarios sin SSO.

13	What-if simulator mejorado	1 sem	Ya hay <code>/intelligence/simulator</code> ; falta save + compare lado-a-lado.
14	Forecasting ML	2 sem	Ya hay <code>forecast/route.ts</code> con proyección lineal; subir a Prophet o LightGBM.
15	DPA + Privacy + SOC2 badges	2-3d UI	Solo es contenido + linking; alto trust con dev mínimo.

● Tier 4 — Estratégico, gran inversión

#	Item	Esfuerzo	Notas
16	AWS integration MVP	6-8 sem	Esperar 5+ clientes Azure validando antes. Scaffolding ya existe (<code>providerFactory</code> , <code>awsFocusMapper</code>).
17	Data residency multi-región	4-6 sem	Solo si entras a EU regulated (GDPR strict).
18	Azure/AWS Marketplace	3-4 sem c/u	Compliance + co-sell; canal de ventas potente.
19	Pen-test + SOC2 Type I	3-6 meses	\$25-50K + auditor; necesario para Enterprise top-tier.

4. Ataque recomendado (próximas 8 semanas)

Semana	Foco
1	Tests + CI (paralelo arrancho)
2	SSO SAML con WorkOS + Notificaciones Slack/Teams/Email
3-4	Paddle real end-to-end + Onboarding wizard
5	Status page + Audit export CSV + API pública v1 (kickoff)
6	Self-service signup + free trial funnel
7-8	API v1 + OpenAPI + Showback PDF + DPA/SOC2 content

Resultado al cabo de 8 semanas: SaaS vendible a Enterprise (SAML + tests + audit + SLA visible) y escalable comercialmente (self-service + billing + notificaciones). AWS y SOC2 quedan como inversión de fase 2 una vez validado el modelo con 5+ clientes en producción.

5. Anexo — Integración AWS (estado y esfuerzo)

Lo que ya está (5%)

- `providerFactory.ts` con stub para AWS (comentado).
- `awsFocusMapper.ts` (56 LOC) — mapper CUR → FOCUS, esqueleto.
- Interfaz `CloudProvider` abstrayendo Azure (preparada para multi-cloud).

Lo que falta

- SDK AWS (`@aws-sdk/client-*`).
- `AWSProvider` real (auth via IAM Role + AssumeRole cross-account).
- CUR (Cost and Usage Report) ingestion desde S3.
- Equivalentes: ResourceGraph (→ Tag Editor / Resource Explorer), Advisor, Reservations Recommender, Trusted Advisor, Carbon footprint, Compute Optimizer.
- DB schema multi-cloud (hoy todo es `subscription_id`; AWS necesita `account_id` + `region`).
- Onboarding UI para AWS (Connect to AWS → bootstrap CloudFormation template).
- Pricing API AWS (Azure Retail Prices ≠ AWS Pricing API).

Esfuerzo realista para MVP AWS

Fase	Semanas	Entregable
------	---------	------------

1 — Auth + Accounts	1	AssumeRole cross-account, CloudFormation template, "Add AWS Account" UI, almacenar credenciales en <code>Credentials</code> extendida
2 — Cost ingestion	1.5	CUR S3 → parseo Parquet → mapear a FOCUS → <code>billing_facts</code> con <code>cloud='aws'</code>
3 — Inventory	1	EC2/EBS/RDS/S3 vía SDK, zombi detection (EBS sin atar)
4 — Recommendations	1	Compute Optimizer + Trusted Advisor + Savings Plans
5 — UI multi-cloud	1.5	Selector cloud en header, dashboards que agreguen Azure+AWS, breakdown por proveedor
6 — Testing + docs	1	Tenant piloto, ajustes, doc onboarding
Total		~7 semanas dev a tiempo completo

Alternativa rápida

Integración con **CloudHealth / Vantage API** como passthrough para mostrar AWS sin reimplementar collectors (3 días, lectura solamente, menos features). Útil como prueba comercial antes de invertir en collector nativo.